

Algorithms for Data Science

Exercise 3

(with Solutions)

See the cheatsheet for valuable remarks!

Task 3.1: Divide and Conquer

Write a recurrence for the running time of the following algorithms in dependence of n . Write it in the form $T(n) = aT(n/b) + f(n)$.

Then solve the recurrences!

(a)

```
StrangeAlgo1(n)
  if n > 1 then
    i = 1
    while i ≤ n do
      i ++
      StrangeAlgo1(n/2)
      StrangeAlgo1(n/2)
      StrangeAlgo1(n/2)
      StrangeAlgo1(n/2)
  return 1;
```

Solution: Recurrence: $T(n) = 4T(n/2) + \Theta(n)$

Solving the recurrence:

We have $a = 4$, $b = 2$, and $f(n) = \Theta(n)$. Thus, $c = \log_b a = \log_2 4 = 2$.

Determining the Case of the Master Theorem:

- Simpler approach with Master Theorem “Lite”:
We have $f(n) = \Theta(n) = \Theta(n^d)$ for $d = 1$. Since $d < c$, Case A holds.
- Alternative approach with normal Master Theorem:
Let $c' = 1 < c$. Since $f(n) = O(n) = O(n^{c'})$, Case A holds.

Final solution: Case A $\rightarrow T(n) = \Theta(n^c) = \Theta(n^2)$.

- (b) What is the running time of the algorithm above if we change the while loop condition from $i \leq n$ to $i \leq n \cdot \log_2 n$?

Solution: Now, the recurrence is $T(n) = 4T(n/2) + \Theta(n \log n)$. (Thus, we have $f(n) = \Theta(n \log n)$ instead of $f(n) = \Theta(n)$.)

Solving the recurrence:

We have the same values for a and b as before, thus, again, $c = 2$.

Case of the Master Theorem:

Since $f(n)$ is not a polynomial, we can't apply Master Theorem "Lite". We have to use the normal Master Theorem. We can't choose $c' = 1$ as before, since $n \log_2 n \neq O(n^1)$ (that is, $f(n)$ grows faster than n). Thus, we need to choose a c' within $1 < c' < 2$. To be on the safe side, let's take a c' very close to c , for instance, $c' = 1.999$. We have

$$n^{c'} = n^{1.999} = n^{1+0.999} = n \cdot n^{0.999} > n \log_2 n .$$

The last inequality holds as $n^{0.999} > \log_2 n$ for sufficiently large n (by the "LOG < POL" Rule from the Cheatsheet of Exercises 2). The inequality $n \log_2 n < n^{c'}$ immediately implies $f(n) = O(n^{c'})$. Hence, we are in case A (as before).

Final solution: Case A $\rightarrow T(n) = \Theta(n^c) = \Theta(n^2)$ (as before!).

Very interesting: The total running time did not change despite the fact that the while loop takes now significantly longer than before.

- (c) And what if we change the while loop condition to $i \leq n^2 / \log_2 n$ instead? Can we apply the Master Theorem?

Solution: The short answer: **no**.

The detailed answer: Now, $T(n) = 4T(n/2) + \Theta(n^2 / \log_2 n)$. The constants a and b are the same before, thus $c = \log_b a$ is still 2. However, $f(n)$ changed to $\Theta(n^2 / \log_2 n)$.

Is it Case B? That is, $f(n) = \Theta(n^c)$? Obviously, we have $f(n) = O(n^c)$ as $n^2 / \log_2 n \leq n^2$ for all $n \geq 2$. Yet, $n^2 \neq O(n^2 / \log_2 n)$ (and thus, $f(n) \neq \Omega(n^c)$) because if $n^2 = O(n^2 / \log_2 n)$ were true, there would be a constant d such that, for all large enough n ,

$$n^2 \leq d \cdot n^2 / \log_2 n \rightarrow \log_2 n \leq d,$$

which, however, contradicts the assumption that d is a constant.

Therefore, only Case A might hold (given $f(n) = O(n^c)$). But does it? Suppose for a moment it does. Thus, there is a constant $c' < c = 2$ such that $f(n) = O(n^{c'})$, that is, there is another constant d such that $n^2 / \log_2 n \leq d \cdot n^{c'}$ for all large n . Let $c'' = c - c'$ and observe that $n^{c'} = n^{c-c''} = n^2 / n^{c''}$. Thus, we have:

$$n^2 / \log_2 n \leq d \cdot n^2 / n^{c''} \tag{1}$$

$$n^{c''} \leq d \cdot \log_2 n . \tag{2}$$

Since $n^{c''}$ is a polynomial (with positive constant degree c'' given $c' < c$), the inequality above contradicts the "LOG < POL" Rule from the Cheatsheet of Exercises 2. Consequently, Case A does not hold, which means, that **no case of the Master Theorem holds!** Thus, we cannot solve the recurrence using the Master Theorem.

So what can we do in such a situation? Luckily, there is an extended form of the Master Theorem (not discussed in our course) that yields the running time $T(n) = \Theta(n^2 \log \log n)$. To solve even more complicated running time recurrences, one can try the very general Akra-Bazzi Theorem.

(d)

```

StrangeAlgo2(n)
  if n > 1 then
    for i = 1 to n do
      for j = 1 to n do
        k = j
        while k ≤ n do
          k = k + 1
      for i = 1 to 8 do
        StrangeAlgo2(n/2)
  return 1;

```

Solution: Recurrence: $T(n) = 8T(n/2) + \Theta(n^3)$

Why $\Theta(n^3)$? Choose $k = k + 1$ as the dominating operation. Observe that it is executed

$$n \cdot \sum_{j=1}^n (n - j + 1) = n \cdot \sum_{j=1}^n j = n \cdot n \cdot (n + 1) / 2 = \Theta(n^3)$$

times in total.

Solving the recurrence:

We have $a = 8$, $b = 2$, and $f(n) = \Theta(n^3)$. Thus, $c = \log_b a = \log_2 8 = 3$.

Case of the Master Theorem:

- Simpler approach with Master Theorem “Lite”:
We have $f(n) = \Theta(n^3) = \Theta(n^d)$ for $d = 3$. Since $d = c$, we are in Case B.
- Alternative approach with the normal Master Theorem:
Since $f(n) = \Theta(n^3) = \Theta(n^c)$, Case B holds.

Final solution: Case B $\longrightarrow T(n) = \Theta(n^c \log n) = \Theta(n^3 \log n)$.

(e)

```

StrangeAlgo3(a[1..n])
  if n < 2 then
    return a[1]
  else
    ℓ = ⌊n/5⌋
    a' = a[1..ℓ]
    b' = a[2..ℓ + 1]
    p = StrangeAlgo3(a')
    a'[1] = p - 1
    return min(StrangeAlgo3(a'), StrangeAlgo3(b'))

```

Solution: Recurrence: $T(n) = 3T(\lfloor n/5 \rfloor) + \Theta(n)$ (simplified: $T(n) = 3T(n/5) + \Theta(n)$)

Solving the recurrence:

We have $a = 3$, $b = 5$, and $f(n) = \Theta(n) = \Theta(n^1)$. Thus, $c = \log_b a = \log_5 3 = 0.68 \dots < 1$. (You don't have to compute the logarithm, you can also use the obvious fact $\log_5 3 < \log_5 5 = 1$.)

Case of the Master Theorem:

- Simpler approach with Master Theorem “Lite”:
We have $f(n) = \Theta(n) = \Theta(n^d)$ for $d = 1$. Since $d > c$ and given that the regularity condition holds by the given assumption(*), we are in Case C.
- Alternative approach with the normal Master Theorem:
Let $c' = 1 > c$. Since $f(n) = \Omega(n^{c'})$ and since the regularity condition holds by assumption(*), we conclude that Case C holds.

Final solution: Case C $\longrightarrow T(n) = \Theta(f(n)) = \Theta(n)$.

(*): A proof of the regularity condition is not required in this task. We are allowed to assume that the condition holds. For completeness, though, here a proof if you don't believe the assumption. The regularity condition holds if there is a constant c'' with $0 < c'' < 1$ such that $c'' \cdot f(n) \geq a \cdot f(n/b)$ (where $f(n)$ is the exact function $f(n) = n$ and not the whole “function class” $\Theta(f(n))$). To check if such a c'' exist, let us first compute $a \cdot f(n/b) = 3 \cdot f(n/5) = 3 \cdot n/5 = 3/5 \cdot f(n) \leq c'' \cdot f(n)$ for $c'' = 3/5 < 1$ as required. Hence, the regularity condition holds.

In fact, the regularity condition automatically holds if $f(n)$ is a polynomial. This is the reason why Master Theorem “Lite” does not ask for the regularity condition in Case C.

Bonus Tasks

Tasks for solving at home (not necessarily discussed in classes); relevant for short tests and the exam.

Task 3.2: Master Theorem

Solve the following recurrences using the master theorem. *First, determine $f(n)$ and the values of a and b . Secondly, compute c . Finally, conclude which case of the master theorem holds (by determining an appropriate c' if Case A holds) and write the solution.*

In the following, you can assume that $f(n)$ satisfies the regularity condition of Case C provided that the first condition of Case C holds.

(a)

$$T(n) = 5T(n/5) + \Theta(\log n)$$

Solution: We have $a = 5$, $b = 5$, and $f(n) = \Theta(\log n)$. Thus, $c = \log_b a = \log_5 5 = 1$. Let $c' = 1/2 < c$. Since $n^{1/2}$ grows faster than $\log n$, we have $f(n) = O(\log n) = O(n^{1/2}) = O(n^{c'})$. Hence, Case A holds. Thus, we have $T(n) = \Theta(n^c) = \Theta(n)$.

(b)

$$T(n) = 3T(n/9) + \sqrt{n} + n^{1/4}$$

Solution: We have $a = 3$, $b = 9$, and $f(n) = \sqrt{n} + n^{1/4} = n^{1/2} + n^{1/4} = \Theta(n^{1/2})$. Thus, $c = \log_b a = \log_9 3 = 1/2$. Since $f(n) = \Theta(n^{1/2}) = \Theta(n^c)$, Case B holds. Thus, we have $T(n) = \Theta(n^c \log n) = \Theta(\sqrt{n} \log n)$.

(c)

$$T(n) = 10^{20}T(\lceil n/100 \rceil) + 2^{\log(n^9)} \cdot \log n$$

Solution: We have $a = 10^{20}$, $b = 100$, and $f(n) = 2^{\log(n^9)} \cdot \log n = n^9 \log n$. Thus, $c = \log_b a = \log_{100} 10^{20} = \log_{100} 10^{2 \cdot 10} = \log_{100} (10^2)^{10} = \log_{100} 100^{10} = 10$. Let $c' = 9.5 < c$. Since $n^{9.5} = n^9 \cdot \sqrt{n}$, it grows faster than $n^9 \cdot \log n$. Thus, $f(n) = O(n^9 \log n) = O(n^{c'})$ and Case A holds. Therefore, we have $T(n) = \Theta(n^c) = \Theta(n^{10})$.

(d)

$$T(n) = n + T(\lfloor n/9 \rfloor)$$

Solution: We have $a = 1$, $b = 9$ and $f(n) = n$. Thus, $c = \log_b a = \log_9 1 = 0$. Let $c' = 1 > c$. Since $f(n) = \Omega(n) = \Omega(n^{c'})$, Case C holds. Since by assumption also the regularity condition holds, we have $T(n) = \Theta(f(n)) = \Theta(n)$.

(e)

$$T(n) = 14n + \sqrt{10000} - 4n + T(\lceil n/10 \rceil) - 10n$$

Solution: After simplifying, we have $T(n) = T(n/10) + 100$. Thus, we have $a = 1$, $b = 10$ and $f(n) = 100 = \Theta(1)$. Thus, $c = \log_b a = \log_{10} 1 = 0$. Since $n^c = n^0 = 1$, we have $f(n) = \Theta(1) = \Theta(n^c)$ and Case B holds. Thus, $T(n) = \Theta(n^c \log n) = \Theta(\log n)$.

Task 3.3: Simplifications of Recurrences

Rewrite the following recurrences in the form $T(n) = aT(n/b) + \Theta(f(n))$ (where a and b are constants) simplifying $f(n)$ as far as possible. You don't have to solve the recurrences.

(a)

$$T(n) = 2T(n/3) + 42n^2 + 100n + T(n/3)$$

Solution: $T(n) = 3T(n/3) + \Theta(n^2)$

Proof: $f(n) = 42n^2 + 100n = \Theta(n^2)$.

(b)

$$T(n) = 13 + 20 \log n + T(n/7) + 2(\sqrt{n} + T(n/7))$$

Solution: $T(n) = 3T(n/7) + \Theta(\sqrt{n})$

Proof: $f(n) = 2\sqrt{n} + 20 \log n + 13 = \Theta(\sqrt{n})$. (Recall that $\sqrt{n}$ grows faster than $\log n$.)

(c)

$$T(n) = \log(n^3) + n^{\log_4 1} \cdot T(n/10) + (\log n)^2 + \log(1024) + \log(\log n)$$

Solution: $T(n) = T(n/10) + \Theta(\log^2 n)$.

Note: $\log^2 n$ is short for $(\log n)^2$, which in turn is short for $(\log n) \cdot (\log n)$.

Proof: First, observe that $\log_4 1 = 0$. Thus, $n^{\log_4 1} = n^0 = 1$. Therefore, $n^{\log_4 1} \cdot T(n/10) = 1 \cdot T(n/10)$. Secondly, recall that $\log(n^3) = 3 \log n$. Thus, $f(n) = \log^2 n + 3 \log n + \log \log n + \log(1024)$. Note that $\log(1024)$ (which resolves to 10) is an uninteresting constant. The fastest growing function is $\log^2 n$. Thus, $f(n) = \Theta(\log^2 n)$ as claimed.

Task 3.4: Divide and Conquer++

For each of the following algorithms, write a recurrence for the running time and a recurrence for the *memory complexity*. Write the recurrences in the form $T(n) = aT(n/b) + \Theta(f(n))$ simplifying $f(n)$ as far as possible. You don't have to solve the recurrences. (For the memory complexity, use M instead of T .)

(a)

```

StrangeAlgo4( $a[1..n]$ )
  if  $n < 2$  then
     $\lfloor$  return  $a[1]$ 
  else
     $\ell = \lfloor n/7 \rfloor$ 
     $a' = a[1..\ell]$ 
     $b' = a[2..\ell + 1]$ 
     $i = 1$ 
    while  $i \leq \ell$  do
       $\lfloor i++$ 
     $k = 1$ 
    while  $k < 7$  do
       $p = \text{StrangeAlgo4}(a')$ 
       $a'[1] = p - 1$ 
       $\lfloor k++$ 
     $i = 1$ 
    while  $i \leq n$  do
       $j = 1$ 
      while  $j \leq n$  do
         $\lfloor j++$ 
         $\lfloor b'[1] = \min(a[i], a[j])$ 
       $i++$ 
    return  $\max(\text{StrangeAlgo4}(a'), \text{StrangeAlgo4}(b'))$ 

```

Solution: Time: $T(n) = 8T(\lfloor n/7 \rfloor) + \Theta(n^2)$ (Note that the second while loop repeats only six times.)

Space: $M(n) = M(\lfloor n/7 \rfloor) + \Theta(n)$ (Note that the $\Theta(n)$ accounts for the memory of the current call and $M(\lfloor n/7 \rfloor)$ accounts for the total memory of the eight recursive calls to **StrangeAlgo4**—note that we didn't write $8 \cdot M(\lfloor n/7 \rfloor)$ as the eight recursive calls can reuse their memory. Why? Observe that we call them one after the other. Thus, if one recursive call ends, its auxiliary variables and arrays are discarded and their memory becomes available for the next recursive call. Consequently, we can count them as one call when computing the memory complexity.)

Note that we can simplify the above recurrences by writing $n/7$ instead of $\lfloor n/7 \rfloor$.

(b)

Input: An array $a[1..n]$ whose all elements are pairwise different and whose length n is a power of 2, i.e., $n = 2^d$ for some integer d .

Output: The array a is sorted increasingly.

```

SlowQuickSort( $a[1..n]$ )
  if  $n < 2$  then
    return  $a$ 
  pivot = 1
   $a1[1..n][1..n]$  new empty two-dimensional array
   $a2[1..n][1..n]$  new empty two-dimensional array
   $p1 = 1$ 
   $p2 = 1$ 
  while pivot  $\leq n$  do
    for  $i = 1$  to  $n$  do
      if  $a[i] < a[pivot]$  then
         $a1[pivot][p1] = a[i]$ 
         $p1 = p1 + 1$ 
      else if  $a[i] \geq a[pivot]$  then
         $a2[pivot][p2] = a[i]$ 
         $p2 = p2 + 1$ 
    if  $p1 == n/2 + 1$  and  $p2 == n/2 + 1$  then
      break // Leaves the while loop immediately
    pivot ++
     $p1 = 1$ 
     $p2 = 1$ 
   $b1[1..p1-1]$  new empty array
   $b2[1..p2-1]$  new empty array
  for  $i = 1$  to  $n/2$  do
     $b1[i] = a1[pivot][i]$ 
     $b2[i] = a2[pivot][i]$ 
   $b1 = \text{SlowQuickSort}(b1)$ 
   $b2 = \text{SlowQuickSort}(b2)$ 
  for  $i = 1$  to  $n/2$  do
     $a[i] = b1[i]$ 
     $a[i + n/2] = b2[i]$ 
  return  $a$ 

```

Solution: Time: $T(n) = 2T(n/2) + \Theta(n^2)$

Space: $M(n) = T(n/2) + \Theta(n^2)$ (the two-dimensional array needs $\Theta(n^2)$ space, the two recursive calls count as one because the memory space of the first call can be reused by the second call.)

(c) What property has the element $a[pivot]$ immediately after the while loop is left?

Solution: It is the $((n/2) + 1)$ smallest element of a (or $(n/2)$ -largest element of a). In other words, it is the larger one of the two middle elements (hence, it is a *median*).

- (d) What happens, if we run `SlowQuickSort(a)` with an array a where all the elements are the same?

Solution: The program never terminates as, in such a case, $b2$ is the same as a (since by assumption all elements are the same and in particular equal to $a[pivot]$, they are put into $a2$ and then to $b2$) and therefore `SlowQuickSort(b2)` is called forever.

Advanced Tasks

Challenging tasks for motivated students, but beyond the scope of this course. Their difficulty level is comparable to that of standard computer science courses.

Task 3.5: Here and There

You are given two arrays $here[1..n]$ and $there[1..n]$ containing numbers from $\{1, 2, 3\}$. There are n smileys numbered from 1 to n . For $1 \leq i \leq n$, $here[i]$ is the initial position of smiley i and $there[i]$ is the position it wishes to be. There are two rules:

1. All smileys on the same position are stacked over each other such that smileys with larger number are below smileys with smaller number.
2. A single step consists of taking one smiley from the top of one position and placing it on the top of some other position.

Write a function `fulfill_wishes` that writes how to fulfill the wishes of all the smileys (at the same time and without violating the rules at any point)! The output of your function should be a sequence of move operations of the form “Take the top-most smiley from position i and place it on top of position j ”.

Solution: (Sketch) The task is a generalization of the known “Tower of Hanoi” problem. Write two auxiliary procedures: `from_here_to_k(m, here[1..n], k)` and `from_k_to_there(m, k, there[1..n])`. The first procedure moves the smileys $1, \dots, m$ from their initial positions to the given position k . The second procedure moves the smileys $1, \dots, m$ from the given position k to their desired positions. Then use both procedures to move all the smileys from *here* to *there*.